🗺️ The Ultimate Git Command Reference

1. Setup & Configuration (Do once per computer)

- git config --global user.name "Your Name" — Tells Git who is making the commits.
- git config --global user.email "you@example.com" — Links your email to your commits.
- git config --list — Displays all your current global settings.

2. Starting a Project

- git init — Turns an ordinary folder into a Git repository (creates the invisible .git tracking folder).
- git clone <url> — Downloads an existing project from GitHub/GitLab onto your computer.

3. The Daily Workflow (Save, Stage, Commit)

- git status — Shows you what files have changed, what is staged, and what branch you are on. *(Use this constantly!)*
- git add <file> or git add . — Moves modified files from your working directory into the **staging area** (preparing them to be saved).
- git commit -m "Message" — Takes everything in the staging area and locks it into Git's history permanently with a descriptive message.
- git diff — Shows the exact line-by-line text changes you made before staging them.

4. Branching & Merging (Working safely in parallel)

- git branch — Lists all local branches and shows which one you are currently on.
- git checkout -b <name> (or git switch -c <name>) — Creates a brand new branch and switches you to it instantly.
- git checkout <name> (or git switch <name>) — Switches you from your current branch to an existing branch.
- git merge <name> — Takes all commits from another branch and pastes/combines them into your current active branch.
- git branch -d <name> — Deletes a branch safely once its work is done.

5. Working with Remotes (GitHub, GitLab, etc.)

- git remote -v — Shows the web address of the online repository connected to your project.
- git fetch — Downloads updates from the online repository *without* changing your local files yet.
- git pull — Downloads updates from the online repository and *automatically merges* them into your current branch.
- git push — Uploads your local commits up to the online repository.

6. Fixing Mistakes & Time Travel

- git log --oneline --graph — Shows a pretty visual timeline of all your past commits.

- git reset <file> — Unstages a file (takes it out of the staging area without losing your edits).
- git stash — Temporarily saves uncommitted work so you can switch branches cleanly, and lets you pop it back later (git stash pop).

1. Repository Setup & Basics

Command	What it does & Why we use it
git init	Initializes an empty Git repository in your current folder, turning on version control tracking.
git status	Shows the state of your working directory (modified files, staged files, and your active branch). *Use this constantly!*
git add <file>	Stages your changes, moving them into the "shopping cart" ready to be committed.
git commit -m "Message"	Permanently saves your staged changes into Git's permanent history with a descriptive message.

2. Branching & Parallel Workflows

Command	What it does & Why we use it
git branch	Lists all local branches in your repository and highlights your current one.
git checkout -b <name>	Creates a brand new branch and switches you to it instantly (isolates new features).

git checkout <name>	Switches you from your current branch to another existing branch.

3. Merging & Conflict Resolution

Command	What it does & Why we use it
git merge <branch>	Combines the commit history of another branch into your currently active branch.
Conflict Markers (<<<<<<<, =======, >>>>>>>)	Text blocks inserted by Git when two branches edit the exact same line differently, allowing you to manually choose the final code.

4. Pull Requests & Collaboration Simulation

Concept / Command	What it does & Why we use it
git diff <branch>	Displays line-by-line differences between your current branch and another branch before merging.
Pull Request (PR)	An online review workflow (on GitHub/GitLab) where team members inspect code changes and approve them before merging into the main branch.

Here are the three essential advanced Git tools that will give you complete control over your history, experimental code, and commit cleanup.

1. git stash (The Temporary Safety Net)

Use git stash when you are in the middle of working on something, but need to quickly switch branches without committing unfinished or broken code.

Bash

```
None
# Save your current dirty work temporarily and clean your working
directory
git stash save "WIP: navbar styling"

# Switch branches, do emergency work, and come back
git checkout master

# Re-apply your stashed changes back onto your working directory
git stash pop
```

- **git stash list**: View all your stored stashes.
- **git stash apply**: Re-applies the stashed changes *without* removing them from the stash list.
- **git stash drop**: Permanently deletes the most recent stash.

2. git reset (Time Travel & Undoing Commits)

Use git reset to move your branch backwards to a previous commit. There are three modes depending on what you want to do with your changes:

Mode	Command	What happens to your files?
Soft	git reset --soft HEAD~1	Undoes the last commit, but **keeps your changes staged** in your working directory.
Mixed (Default)	git reset HEAD~1	Undoes the last commit and **unstages the files**, keeping your code uncommitted.
Hard	git reset --hard HEAD~1	**Wipes out the last commit and permanently deletes all changes** made in it.

Danger Zone
git reset --hard is destructive. Any uncommitted work or reset commits will be permanently lost unless you recover them via git reflog.

3. git rebase (Cleaner, Linear History)

Instead of creating a messy merge commit when combining branches, git rebase takes your feature branch commits, temporarily detaches them, applies the latest master commits, and then replays your commits right on top.

Bash

```
None
# While on your feature branch:
git rebase master
```

Interactive Rebase (git rebase -i)

Use interactive rebase to clean up, combine (squash), or reword your commits before opening a Pull Request:

Bash

```
None
git rebase -i HEAD~3
```

This opens a text editor showing your last 3 commits with actionable directives:

- **pick**: Keep the commit as-is.
- **reword**: Keep the commit, but edit the commit message.
- **squash**: Combine this commit into the previous commit (great for cleaning up "fix typo" commits!).